SmartPlanif — Pseudo-code complet

Allocation de bus, chauffeurs et receveurs aux voyages (C# / .NET 8, Clean Architecture, Blazor). Ce document décrit, couche par couche, la logique de la dernière version du code, sans les détails de syntaxe.

1. Vue d'ensemble

```
Web (Blazor)  ->  Application (cas d'usage, ports)  ->  Domain (règles,
algorithmes)
      \                                                    ^
       \-> Infrastructure (MySQL/Dapper, démo, JSON) -------/  (implémente
les ports)
```

Flux principal « allouer une journée » :

```
CHARGER (trips, voyages verrouillés, ressources, politique)
  -> DIAGNOSTIC de faisabilité (phase 0)
  -> CONSTRUCTION (glouton ou par phases)
  -> AMÉLIORATION (ALNS + recuit simulé, optionnelle)
  -> COMPLÉTION déterministe
  -> SCORES EXACTS + CONTRÔLE INDÉPENDANT
  -> ENREGISTREMENT transactionnel (si garde-fous OK)
```

2. Domaine : entités et types

```
TYPE AgentRole = Driver | Receiver | Both | Other
  CanDrive(r)   = r est Driver ou Both
  CanReceive(r) = r est Receiver ou Both
  IsPlannable(r)= r != Other

RECORD Agent(Id, Name, Role, RestDay?, CenterId?, DelegationId?)
RECORD Bus(Id, Plate, Active, Seats, Standing, CategoryId?, CenterId?,
DelegationId?, InServiceSince?)
  Capacity = Seats + Standing
RECORD Trip(Id, LineId, LineLabel, LineTypeId?, Departure, Arrival,
DistanceKm,
            CenterId?, DelegationId?, OriginStationId?,
DestinationStationId?, ExpectedPassengers?)
  Interval = [Departure, Arrival[
  WorkDate = date(Departure)
RECORD Unavailability(Kind, ResourceId, Period, Reason)
```

```
RECORD AssignedTrip(Trip, BusId, DriverId, ReceiverId, IsRealized)
RECORD TripAssignment(Trip, Bus, Driver, Receiver, Score, MaxScore,
Justification)

CLASSE ResourcePool(Buses, Agents, Unavailabilities)
  AllAgentsInterchangeable = TOUS les agents planifiables ont le rôle Both
  UnavailabilityFor(kind, id) = périodes d'indisponibilité de cette ressource

TimeInterval [Start, End[ :
  Overlaps(o) = Start < o.End ET End > o.Start     // bout à bout = pas de
chevauchement
```

3. Politique et catalogue de contraintes

```
CLASSE AllocationPolicy
  MaxDailyWork=8h, MaxAmplitude=12h, MinAgentRest=30min,
MinBusTurnaround=15min
  MaxTripsPerAgentPerDay=6, MaxBusKmPerDay=500, RestSaturation=4h
  Constraints : dictionnaire code -> (Mode, Poids)
  CategoryRules : liste (LineTypeId, Allowed[], Preferred[])
  Alns : paramètres (iterations, secondes, température, refroidissement,
graine, bruit, pénalité, équité...)

Mode = Disabled | Hard | Soft

CATALOGUE : liste ordonnée de définitions
  Chaque définition = fiche (code, nom, modes supportés, défaut, poids,
verrouillée?)
                    + fabrique de règle dure (optionnelle) + fabrique de
règle souple (optionnelle)
  Profils : LockedHard | HardToggle | HardOrSoft | SoftToggle

FONCTION ResolveSetting(policy, fiche)
  SI fiche.verrouillée        ALORS RETOURNER (fiche.modeDéfaut,
fiche.poidsDéfaut)
  SI policy.Constraints[code] existe ET son mode est supporté
       ALORS RETOURNER (mode configuré, max(0, poids))
  RETOURNER valeurs par défaut

FONCTION BuildRuleSet(définitions, policy)
  hard = [] ; soft = []
  POUR chaque définition (dans l'ordre : les plus économiques d'abord)
     s = ResolveSetting(policy, définition.fiche)
     SI s.Mode = Hard  ET fabrique dure existe          ALORS
hard.ajouter(fabrique dure())
```

```
     SI s.Mode = Soft  ET fabrique souple existe ET s.Poids > 0
                                                          ALORS
soft.ajouter(fabrique souple(s.Poids))
  RETOURNER RuleSet(hard, soft)       // MaxScore = somme des poids souples
```

Contraintes **bus** : B02 actif, B03 pas de chevauchement (verrouillées) ; B10 maintenance, B09 remise en service, B05 capacité, B11 km/jour (dures désactivables) ; B06 catégorie, B07 centre, B08 délégation (dure ou souple) ; BP03 à BP08 (souples). Contraintes **agents** : A05 pas de chevauchement (verrouillée) ; A03 jour de repos, A04 congés, A07 repos minimal, A08 heures max, A09 voyages max, A10 amplitude (dures désactivables) ; A11 centre, A12 délégation (dure ou souple) ; AP03 à AP06 (souples).

4. Règles dures (un rejet est définitif)

```
Interface HardRule<T> : Check(contexte) -> Violation | null
Contexte = (ressource, voyage, planning de la ressource, indisponibilités,
politique)

B02 BusActive          : violation SI NON bus.Active
B05 BusCapacity        : violation SI voyage.ExpectedPassengers connu ET
bus.Capacity < passagers
B11 BusDailyKm         : violation SI planning.KmOn(jour) + voyage.DistanceKm
> MaxBusKmPerDay
B06 CategoryCompat     : r = règle du type de ligne ; SI r absente OU Allowed
vide -> OK
                         violation SI catégorie du bus NON dans Allowed
A03 RestDay            : violation SI agent.RestDay = jour(départ) OU
jour(arrivée)
A08 DailyWork          : violation SI travaillé(jour) + durée(voyage) >
MaxDailyWork
A09 MaxTripsPerDay     : violation SI nbVoyages(jour) >=
MaxTripsPerAgentPerDay
A10 Amplitude          : violation SI amplitude(jour, avec le voyage) >
MaxAmplitude
A05/B03 NoOverlap      : violation SI planning.Overlaps(voyage.Interval)
A04/B10 Unavailability : violation SI une indisponibilité chevauche le voyage
A07/B09 MinGap         : prev = dernier voyage terminé avant le départ
                         SI prev existe ET départ - prev.Arrivée < min ->
violation
                         next = premier voyage commençant après l'arrivée
                         SI next existe ET next.Départ - arrivée < min ->
violation
```

```
SameValue (B07/B08/A11/A12 en mode dur) :
                          violation SI les deux valeurs sont connues ET
différentes
                          (valeur inconnue = bénéfice du doute, pas de rejet)
```

5. Règles souples (valeur dans [0,1] x poids)

Convention : 1 = idéal, 0 = pire cas, 0,5 = donnée inconnue (neutre).

```
Match (B07,B08,A11,A12) : 0,5 si inconnu ; 1 si égal ; 0 sinon
FewTrips (BP07,AP05)    : 1 / (1 + nbVoyages(jour))
CategoryAllowed (B06 souple) : 0,5 si aucune règle ; 1 si autorisée ; 0 sinon
CategoryPreference (BP03)    : 0,5 si pas de préférée ; 0 si catégorie
inconnue
                               1 si préférée ; 0,5 si simplement autorisée ;
0 sinon
CapacityFit (BP04)  : passagers connus ? clamp(passagers / capacité, 0, 1) :
0,5
StationContinuity (BP05) : prev = voyage précédent ; 0,5 si inconnu
                           1 si prev.Destination = voyage.Origine ; sinon 0
LowKm (BP06)        : 1 - clamp(kmDuJour / MaxBusKmPerDay, 0, 1)
NewerBus (BP08)     : 0,5 si date inconnue ; 1 - clamp(âge / 20 ans, 0, 1)
RestTime (AP03)     : 1 si aucun voyage précédent ; clamp((départ -
prev.Arrivée) / RestSaturation, 0, 1)
LowWorkedHours (AP04): 1 - clamp(travaillé(jour) / MaxDailyWork, 0, 1)
SameLineContinuity (AP06): 0,5 si pas de précédent ; 1 si même ligne ; sinon
0

Points d'un critère = poids x valeur ;  Score d'un candidat = somme des
points
```

6. Plannings en mémoire

```
CLASSE ResourceSchedule  (liste de voyages triée par départ)
  Add(v) : ajouter puis trier ;  Remove(v) : retirer par Id ;  Clone() :
copie de la liste
  Overlaps(intervalle), Previous(t), Next(t)
  WorkedOn(jour), TripCountOn(jour), KmOn(jour), TotalWorked()
  SpanOn(jour, candidat) : min des départs -> max des arrivées, candidat
inclus

CLASSE ScheduleBook : un planning par bus, UN SEUL par agent (chauffeur ou
receveur)
  Commit(v, bus, chauffeur, receveur) : ajoute v aux trois plannings
  Remove(...)                         : opération inverse
  Clone()                             : copie profonde (indispensable pour
l'ALNS)
  // un planning par agent => jamais chauffeur ET receveur en même temps

CLASSE AllocationState(tripsÀPlanifier, book, affectations{tripId ->
TripAssignment})
  Assign(a)   : refuse si déjà affecté ; book.Commit ; enregistre
  Unassign(id): retire l'affectation ; book.Remove ; faux si absente
  UnassignedTrips = voyages sans affectation
  Clone()     : copie profonde (book + dictionnaire)
```

7. Décision pour UN voyage

```
FONCTION Select(candidats, voyage, règles, policy, bruit?) -> (classés,
rejetsParCode)
  POUR chaque candidat
     v = première règle dure violée (dans l'ordre du catalogue)
     SI v existe : rejets[v.code]++ ; CONTINUER
     détails = pour chaque règle souple : (code, poids, clamp(Evaluate, 0,
1))
     score   = somme des points
     clé     = score + (aléa dans [-1,1]) x bruit x règles.MaxScore    //
bruit seulement pour l'ALNS
  TRIER par clé décroissante, puis Id croissant (déterminisme)

FONCTION TryAssign(voyage, pool, book, explain?, aléa?)
  bruit = aléa présent ? Alns.InsertionNoise : 0
  bus      = Select(tous les bus, ...).meilleur
  chauffeurs = Select(agents aptes à conduire, ...)
  receveurs  = pool.AllAgentsInterchangeable ? chauffeurs    // classement
```

```
calculé une seule fois
                                                : Select(agents aptes receveur,
...)
  chauffeur = chauffeurs.meilleur
  receveur  = premier de receveurs DIFFÉRENT du chauffeur
  SI bus, chauffeur ou receveur manquant
       RETOURNER échec (rôles manquants + rejets cumulés par code de règle)
  RETOURNER affectation (score = bus + chauffeur + receveur, justification si
explain)
  // TryAssign ne modifie JAMAIS les plannings : l'appelant doit faire Assign

FONCTION CountFeasibleOptions(voyage, pool, book)       // difficulté du
voyage
  buses = nombre de bus éligibles (règles dures)
  D = ids de chauffeurs éligibles ; R = ids de receveurs éligibles
  agents = min( |D|, |R|, |D ∪ R| / 2 )                 // il faut deux
personnes distinctes
  RETOURNER min(buses, agents)                           // petit = voyage
très contraint
```

8. Faisabilité et construction du plan initial

```
FONCTION PeakConcurrent(voyages, écartSupplémentaire)
  événements = pour chaque voyage : (départ, +1) et (arrivée + écart, -1)
  trier par temps, les -1 avant les +1 à égalité
  RETOURNER le maximum du cumul courant

FONCTION CountCompatibleBuses(pool, policy, typeLigne)
  actifs = bus actifs
  SI B06 n'est pas dure OU pas de règle OU Allowed vide -> |actifs|
  SINON -> nombre d'actifs dont la catégorie est dans Allowed

FONCTION Analyze(voyages, pool, policy, date, libellés)        // phase 0
  ligne « Bus » : demande = PeakConcurrent(tous, MinBusTurnaround) vs bus
actifs
  POUR chaque type de ligne : demande = PeakConcurrent(groupe) vs
CountCompatibleBuses
  disponibles = agents planifiables hors jour de repos
  SI agents interchangeables :
       demande simultanée = 2 x PeakConcurrent(tous, MinAgentRest) vs
disponibles
       demande en heures  = ceil(2 x total des heures) vs floor(disponibles x
MaxDailyWork)
  SINON : mêmes calculs séparément pour chauffeurs et receveurs
  chaque ligne : « Suffisant » si demande <= offre, sinon « INSUFFISANT »
```

```
CONSTRUCTION GLOUTONNE (référence)
  état = AllocationState(voyages, book verrouillé)
  POUR chaque voyage trié par (départ, Id)
     r = TryAssign(...) ; SI affectation ALORS état.Assign

CONSTRUCTION PAR PHASES
  lots = voyages groupés par type de ligne ; pour chaque lot :
     pic = PeakConcurrent(lot, MinBusTurnaround)
     compatibles = CountCompatibleBuses(...)
     tension = pic / max(1, compatibles)
  trier les lots par tension décroissante (puis taille décroissante, puis
clé)
  POUR chaque lot, sur le MÊME état partagé :
     ordonner ses voyages par difficulté croissante = CountFeasibleOptions,
puis départ, puis Id
     POUR chaque voyage : TryAssign ; SI affectation ALORS état.Assign
     enregistrer une ConstructionPhase (type, nb voyages, pic, bus
compatibles, tension, affectés)
```

9. Fonction objectif (à minimiser)

```
FONCTION Evaluate(état)
  déficit  = somme sur chaque affectation de (MaxScore - Score)       //
scores à l'insertion (approximation)
  équité   = écart-type des heures totales travaillées par les agents
planifiables
  RETOURNER  UnassignedPenalty x nbVoyagesNonAffectés        // 100 000 :
domine tout
           + déficit
           + FairnessWeight x équité                          // 10
// Un meilleur score n'est jamais obtenu en perdant un voyage.
```

10. Amélioration : ALNS avec recuit simulé

```
FONCTION Improve(initial, pool, allocator, progress, annulation)
  rng = Random(graine)                                      // même graine =
même résultat
  poidsDestruction[4] = 1 ; poidsRéparation[3] = 1
  courant = initial.Clone() ; meilleur = initial.Clone()
  valCourante = valMeilleure = Evaluate(courant)
  T = InitialTemperature
```

```
  TANT QUE  itération < MaxIterations
        ET  temps < MaxSeconds
        ET  sansAmélioration < StagnationLimit
        ET  pas d'annulation demandée
     itération++
     d = roulette(poidsDestruction, rng) ; r = roulette(poidsRéparation, rng)

     candidat = courant.Clone()                          // le courant reste
intact si refus
     nb = ChooseDestroyCount : fraction aléatoire entre DestroyFractionMin et
Max
          des voyages affectés, bornée à [min(2, affectés), affectés]
     opérateurDestruction[d].Destroy(candidat, nb, rng)
     opérateurRéparation[r].Repair(candidat, contexte)    // toujours en
respectant les règles dures
     valCandidat = Evaluate(candidat)

     récompense = 0
     SI valCandidat < valMeilleure - ε :
          meilleur = candidat.Clone() ; valMeilleure = valCandidat
          courant = candidat ; récompense = 33 ; sansAmélioration = 0
     SINON :
          sansAmélioration++
          SI valCandidat < valCourante - ε : courant = candidat ; récompense
= 9
          SINON SI rng.NextDouble() < exp( -(valCandidat - valCourante) /
max(T, 1e-9) ) :
                                        courant = candidat ; récompense = 3
// accepter une dégradation

     // adaptation des poids : moyenne mobile exponentielle
     poids[op] = max(0,1 ; (1 - réaction) x poids[op] + réaction x
récompense)
     T = T x CoolingRate
     émettre convergence et progression à intervalles réguliers

  RETOURNER (meilleur, itérations, durée, objectifs initial/final, courbe,
statistiques par opérateur)
```

Opérateurs de destruction (ils retirent des affectations et libèrent les ressources) :

```
RandomRemoval     : mélanger les voyages affectés, en retirer nb
WorstScoreRemoval : trier par (MaxScore - Score) x (0,5 + aléa) décroissant,
retirer les nb premiers
TimeWindowRemoval : pivot = départ d'un voyage NON affecté (70 %) ou d'un
voyage affecté au hasard
                    retirer les nb voyages affectés dont le départ est le
plus proche du pivot
ResourceRemoval   : jusqu'à 10 tentatives : choisir un voyage graine et un
rôle (bus/chauffeur/receveur)
                    retirer les voyages qui partagent cette même ressource
```

Opérateurs de réparation (réinsèrent les voyages en attente via TryAssign) :

```
ChronologicalRepair      : ordre par (départ, Id), sans bruit
MostConstrainedFirstRepair: ordre par CountFeasibleOptions croissant, avec
bruit
RandomOrderRepair        : ordre mélangé, avec bruit
```

11. Complétion, scores exacts et contrôle indépendant

```
FONCTION TryCompleteUnassigned(état, pool, allocator)
  trier les voyages non affectés par difficulté croissante
  POUR chacun : TryAssign déterministe ; SI affectation ALORS Assign
  RETOURNER le nombre ajouté

FONCTION BuildFinalAssignments(état, pool, allocator)       // scores EXACTS
  book = copie du planning
  POUR chaque affectation triée par (départ, Id)
     POUR chacun de bus, chauffeur, receveur :
        retirer temporairement le voyage de son planning
        évaluer les critères souples dans le même contexte qu'au choix ;
score + texte
        remettre le voyage
     Score = somme des trois ; Justification = « Bus #3 : 87,5 pts [B07=40,
BP04=10] | ... »

FONCTION Validate(état, pool, allocator)                    // indépendant de
l'algorithme
  POUR chaque affectation :
     SI chauffeur ne peut pas conduire  -> violation A02
     SI receveur ne peut pas receveur   -> violation A02
```

```
      SI chauffeur = receveur            -> violation A06
      POUR bus, chauffeur, receveur : retirer le voyage, rejouer TOUTES les
règles dures, remettre
  RETOURNER les violations (une violation = un bug du moteur)
```

12. Cas d'usage « allouer une journée » (couche Application)

```
FONCTION HandleAsync(commande, progress, annulation)
  calendrier.EnsureEditable(date)            // lève PastDayException si la
date < aujourd'hui
  politique  = commande.PolicyOverride OU settings.Load()
  joursVoyages = trips.GetTripsOfDay(date)
  affectés     = trips.GetAssigned(date-1, date+1)          // fenêtre J-
1..J+1, chargée une seule fois
  entrées      = DayPlanningInputs.Resolve(joursVoyages, affectés,
commande.ExistingPlan)
  pool         = resources.GetPool(date-1, date+1)
  rapport = EXÉCUTER EN ARRIÈRE-PLAN Compute(...)
  SI commande.Save != None : rapport.Save = SaveAsync(...)
  RETOURNER rapport

FONCTION Resolve(voyagesDuJour, affectésFenêtre, mode)
  SI mode = ReplaceDay :
       réalisés = affectés du jour avec IsRealized
       verrouillés = affectés hors du jour OU réalisés
       àPlanifier  = voyages du jour sauf réalisés
  SINON (FillMissingOnly) :
       àPlanifier  = voyages du jour absents de « affectés »
       verrouillés = tous les affectés

FONCTION Compute(...)
  allocator = TripAllocator(politique)
  book = ScheduleBook ; POUR chaque verrouillé : book.Commit(...)    // ils
occupent leurs ressources
  faisabilité = Analyze(...)
  stratégie = méthode = Greedy ? Chronologique : Par phases
  construction = stratégie.Build(...) ; objectifConstruction = Evaluate
  SI méthode = PhasedConstructionWithAlns ET il y a des voyages :
       résultat = Improve(état, ...) ; état = résultat.meilleur
  TryCompleteUnassigned(état) ; objectifFinal = Evaluate
  affectationsFinales = BuildFinalAssignments ; violations = Validate
  échecs = pour chaque voyage non affecté, TryAssign donne les causes de
rejet
  RETOURNER AllocationReport (compteurs, objectifs, codes de règles, phases,
faisabilité, ALNS, ...)
```

```
FONCTION SaveAsync(commande, rapport)       // garde-fous : refuser SANS rien
écrire si
  violations > 0                                    -> « contrôle indépendant
: violations »
  rapport.TotalTrips = 0                            -> « rien à enregistrer »
  aucune affectation                                -> « aucun voyage affecté
»
  Save = CompleteDayOnly ET non affectés > 0        -> « journée incomplète »
  SINON assignments.SaveDayPlan(plan, clearUnassigned = (mode = ReplaceDay))
```

13. Infrastructure (MySQL / Dapper)

```
Lecture des voyages (une requête par jour)
  jour de service = trips.DEDATED (à défaut la date de TIME_DEPART)
  JOIN drligne (ligne active si RequireActiveLine) ; filtre ETAT exclus si
configuré
  arrivée = MAX(stop_times.ARRIVAL_TIME) ; distance =
MAX(SHAPE_DIST_TRAVELED) sinon drligne.DENBRKM
  origine/destination = première/dernière station de stop_times
  ressources prévues *_PR et réelles *_RE : NULLIF(x, 0)      // 0 = « aucune
ressource »
  SI arrivée absente (voyages futurs) :
       durée = ItineraryDuration.Resolve(dritin de la ligne) ; arrivée =
départ + durée
  Normalize(horaires) : SI la date du départ est à plus d'un jour du jour de
service (date factice)
       -> remplacer la date par le jour de service, décaler l'arrivée de la
même durée
       SI arrivée <= départ -> +1 jour ; SI arrivée absente ou incohérente ->
voyage écarté

FONCTION GetAssigned(du, au)
  POUR chaque jour : SI bus, chauffeur ET receveur RÉELS tous renseignés ->
AssignedTrip(IsRealized = vrai)
                     SINON SI bus, chauffeur ET receveur PRÉVUS tous
renseignés -> AssignedTrip(IsRealized = faux)

ItineraryDuration.Resolve(arrêts, mode)
  minutes = DEDURTR triées par séquence (non nulles)
  cumulé = max(minutes) ; somme = sum(minutes) ; vitesse = kmMax / (durée/60)
  MaxCumulative -> cumulé ; SumOfLegs -> somme
  Auto : « cumulé » si les valeurs croissent strictement (zéros de tête
ignorés)
         la vitesse plausible (12 à 90 km/h) tranche ; si les deux sont
```

```
plausibles,
         choisir celle la plus proche de 35 km/h

Ressources
  bus : DEACTIF selon BusActiveMode (Auto : tous actifs tant qu'aucun bus n'a
DEACTIF = 1)
  agents : rôle = DefaultRole (Both) sauf exceptions de DECQUAL ; DECJOUR
1..7 -> DayOfWeek (modulo 7)
  indisponibilités : lues seulement si la table resource_unavailability
existe

Écriture du plan (SaveDayPlan) : UNE transaction, tout ou rien
  UPDATE trips SET BUS_PR, CHAUFF_PR, REC_PR par voyage  WHERE ID = x ET
voyage NON réalisé
  SI nbMisesÀJour != nbVoyages -> ROLLBACK (voyage supprimé ou déjà effectué)
  SI ClearUnassigned : remettre à NULL BUS_PR/CHAUFF_PR/REC_PR des voyages
non affectés (non réalisés)
  COMMIT ; exception base (clé étrangère...) -> ROLLBACK, rien n'est écrit
  // les colonnes *_RE ne sont JAMAIS modifiées
```

14. Services annexes et interface

```
PlanningCalendar : Today = date de test (Planning:Today) sinon date locale
  IsEditable(d) = d >= Today ; EnsureEditable(d) lève PastDayException (règle
côté serveur)

AllocationSettingsService.Save(politique)
  erreurs = PolicyValidator.Validate(politique)   // durées > 0, amplitude >=
heures max,
  SI erreurs : refuser                            // poids >= 0, souple sans
poids = erreur,
  SINON store.Save(politique)                     // préférée doit être
autorisée, bornes ALNS...
  Reset : retour à la politique par défaut

PreviewDay : même chargement que l'allocation, mais seulement Analyze +
volumes, aucune écriture

DatabaseDiagnostics.Run(date)   // lecture seule, à lancer EN PREMIER sur des
données réelles
  compare la base à ce que le moteur charge, produit des constats ok / info /
warn / error :
  voyages absents, lignes inactives, arrivées déduites, voyages écartés,
DEACTIF non renseigné,
  jours de repos par défaut (> 50 % identiques -> conseiller de désactiver
```

```
A03),
  nombre d'agents aptes, tables optionnelles absentes

SavedPlanService : Get(date) lit le plan *_PR ; Clear(date) exige un jour
modifiable,
  vide BUS_PR, CHAUFF_PR, REC_PR des voyages non réalisés

Pages Blazor : Contraintes (mode + poids par règle), Types de ligne
(catégories autorisées/préférées),
  Paramètres (8 h, amplitude, repos, ALNS), Diagnostic, Allocation (aperçu,
lancement, progression
  en direct, plan, échecs, violations, bouton Arrêter), Plan enregistré
(relire / effacer)

Program : chaîne de connexion vide -> infrastructure en mémoire (démo) ;
renseignée -> MySQL Transport_Ai
```